

Extended Target Tracking and Classification Using Neural Networks

Barkın Tuncer, Murat Kumru, Emre Özkan
Department of Electrical and Electronics Engineering
Middle East Technical University
Balgat 06800, Ankara, Turkey
{barkin.tuncer, kumru, emreo} @ metu.edu.tr

Abstract—Extended target/object tracking (ETT) problem involves tracking objects which potentially generate multiple measurements at a single sensor scan. State-of-the-art ETT algorithms can efficiently exploit the available information in these measurements such that they can track the dynamic behaviour of objects and learn their shapes simultaneously. Once the shape estimate of an object is formed, it can naturally be utilized by high-level tasks such as classification of the object type. In this work, we propose to use a naively deep neural network, which consists of one input, two hidden and one output layers, to classify dynamic objects regarding their shape estimates. The proposed method shows superior performance in comparison to a Bayesian classifier for simulation experiments.

Index Terms—Extended Target Tracking, Contour Representation, Shape-based Classification, Gaussian Process, Artificial Neural Network, Classification, Deep Learning

I. INTRODUCTION

Current advances in intelligent systems, automated vehicles and unmanned aerial vehicles brought the necessity of short-range tracking systems. In contrast to regular long-range counterparts, it is possible to acquire multiple measurements from an object of interest at each instance using short-range sensors. Therefore, they enable us to extract valuable information to a greater extent related to the object contour along with the kinematics of the object, e.g., position, velocity and orientation. In this regard, extended target/object tracking (ETT) algorithms have provided systematic ways to process these measurements to estimate the kinematic state of the object together with its shape. Algorithms presuming simple shape models, such as circle, rectangle, line, are developed in [1]–[3]. In this branch of ETT algorithms, the most common approach is to utilize random matrix models, where the target extent is represented by an ellipse [4]–[7]. In another line of research, approximate non-parametric models are used to describe the target extent. These methods can simultaneously track and learn various shapes without assuming predefined extents. Random hypersurface models, as suggested in [8], [9], are examples of this class. More recently, algorithms relying on a Gaussian Process (GP) representation of the unknown target extent have been suggested in [10]–[12].

Classification of objects while tracking has been a long-standing problem in the literature. Various algorithms have been proposed to tackle the identification of targets based on their dynamic behavior, motion cues, attributes, fingerprints,

etc. [13]–[15]. One of the early works on *joint target tracking and classification* (JTC) was presented in [16]. Their method aims to compute the joint target state-class posterior density and allows for cross-coupled feedback between state and class. In [15], authors proposed a particle filter based method, which covers the state and feature space, designed for each class. In [17], authors presented an approach to JTC problem based on belief functions. In another fold of studies, [18] proposed a batch iterative optimization algorithm which minimizes the Bayes risk involving classification and estimation errors. A recursive version of this method is introduced in [19]. Recently, the authors of [20] also tackled the extended object classification problem by relying on a Bayes risk.

In most of the aforementioned methods, there exists a connection between the classification result and the tracking filter, e.g., the result of the classification manipulates the tracking filter. On the contrary, the method proposed in this study considers a rather weak coupling between the tracking and classification tasks, since the output of the tracker is used for classification purposes while there is no established feedback mechanism.

In this study, we consider the problem of online classification of dynamic objects using point cloud data. A block diagram of the proposed method is depicted in Figure 1. At each instant, point measurements originated from an object are first processed by GP based extended target tracker (GP-ETT) algorithm which produces estimates of the kinematic state and the contour of the object. After that, the estimated contour is utilized to extract descriptive features of the object shape. These features are then fed to a neural network (NN) model to compute the class probabilities of the object of interest. Note that we recently addressed the problem of tracking and classification in [21]. The method in [21] essentially relies on a similar architecture which exploits the outputs of GP-ETT for classification; however, it carries out the classification by a Bayesian classifier. Therefore, the contribution of the current study is twofold: retaining the basic structure of our previous study we hereby demonstrate the modularity of the proposed framework, and secondly, we also improve the resulting classification accuracy by utilizing an NN-based classifier.

As a final note, although GP-ETT is employed as the tracker in this work, it can be simply generalized to other ETT

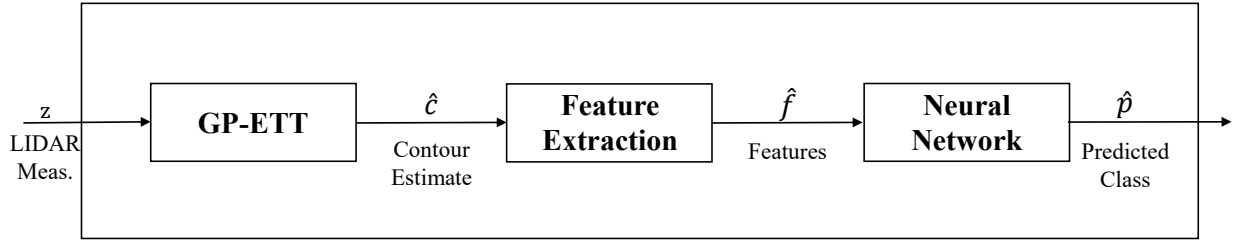


Fig. 1: Block diagram of the proposed online classification algorithm.

algorithms that are capable of generating extent estimates of the objects.

The organization of the paper is as follows. In Section II, we discuss the feature selection process and the details of the NN model. Subsequently, we briefly introduce the GP-ETT algorithm in Section III to provide an insight into the contour representation that we rely on for classification. This is followed by the demonstration of the classification performance via simulation results in Section IV. Finally, we conclude the paper in Section V.

II. OBJECT CLASSIFICATION

In this work, we aim at achieving object classification by using contour estimates produced by an extended object tracker. With this purpose, the estimated contour is to be transformed into some features to acquire a descriptive representation of the object shape. Thereafter, an NN-based classifier will perform the classification task regarding these features. In this section, the details of the selected features and the classifier architecture are discussed.

A. Feature Selection for Contour Representation

In literature, studies on shape classification mostly rely on two different interpretations of the object shape. These interpretations are embodied by either region-based descriptors [22] or contour-based descriptors [23], [24]. Considering substantial amount of empirical evidence, an object can be described as a combination of a set of regions or may be a single body. These regions might include some holes inside. Region-based descriptors make use of all information constituting the shape including the holes or several disjoint regions. On the other hand, contour-based descriptors consider the characteristic shape features extracted from the contour of an object while ignoring what is inside the contour. In this study, we naturally direct our attention to the utilization of the contour-based descriptors since we make use of contour estimates obtained by the tracking algorithm and none of the shapes consist of any holes or disjoint members.

There are various contour descriptors proposed in the literature. The most common ones are the *Fourier Descriptors* (FD), [25], [26], and *Curvature Scale Space* (CSS), [27], [28].

FD consist of the Fourier coefficients of 2-D shapes which can be represented in terms of different shape signatures, such as the contour coordinates expressed as complex numbers or the radial distance between the contour and the object

center, [25]. FD are robust to rotation and affine transformation, while being efficient in terms of computation. CSS, on the other hand, considers the inflection points as descriptors which represent the location of change in the direction of the curvature. The contour is convolved with multiple Gaussian kernels with different standard deviations, also called as width, to smoothen the contour. The inflection points are calculated for different Gaussian kernels and used as descriptors. One of the main advantages of CSS is that it is noise invariant since the characteristic inflection points remain available after filtering with large width kernels, while other points resolve.

In addition to these prominent descriptors, there are also numerous geometric features with various levels of complexity, [29], [30], [31]. These features are basically employed to map object contours into some descriptive representations. Selecting a proper set of features among numerous alternatives is of paramount importance for classification performance. In this regard, we restrict our scope to the simple geometric features to be able to form a basis for computationally efficient and fast operation. The following set of six features are chosen to be respected by the classifier: *elongation* [30], *rectangularity* [29], *circularity* [29], *solidity* [29], *compactness* [31], and *area*. Note that this particular group of features is not hand-crafted to optimize the resulting classification performance. Instead, it is empirically observed to be sufficient for the proof of concept. However, for a specific application, it can be selected to account for any prior knowledge or can be learned in an automated fashion by employing various tools, such as convolutional neural networks, autoencoders.

Elongation is uniquely defined as the ratio of the major and minor axes of the minimum bounding rectangle (which are denoted as l_1 and l_2 in Fig. 2). Rectangularity is the ratio of the object area to the area of the minimum size rectangle that encloses the contour. In Fig. 2, rectangularity can be computed as the object area to the area encapsulated by the orange contour. On the other hand, circularity is related to the ratio of the object area to the area of the minimum confining circle, and it is basically used to measure the similarity of the object of interest to a circle. The minimum bounding circle is illustrated by the green line in Fig. 2, and the circularity is calculated as the ratio of the object area to the area enclosed by the green contour. Solidity is an indicator of the shape being convex or concave. It is defined as the ratio of the object area to the area of the convex hull. The convex hull of the shape is represented

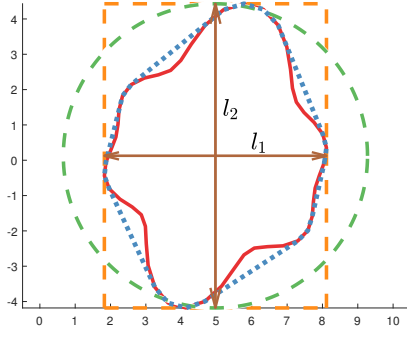


Fig. 2: Visualization of the parameters used in the feature extraction process. The solid red curve represents the contour of the object. The minimum bounding circle and rectangle are plotted in green and yellow, respectively. Major and minor axes of the minimum bounding rectangle are denoted by l_1 and l_2 . The blue dashed line indicates the convex hull of the object.

by the blue line in Fig. 2. Lastly, compactness measures the contour complexity versus the enclosed area. The value of compactness increases with increasing shape complexity. The expression to calculate the compactness is given as [31]

$$C = 1 - \frac{4\pi a}{p^2}, \quad (1)$$

where a and p are the area and the perimeter of the shape, respectively.

The computed features of the object contour are then passed to the NN-based classifier. The classifier architecture is revealed in the following subsection.

B. Classifier Architecture

In this study, we realize object classification by a naively deep feedforward NN. A typical NN model is basically a collection of special processing units, called neurons, which are grouped into different layers, such as input, hidden and output layers. Specifically, in a feedforward NN, there exist weighted connections between these layers while there is no connection between neurons within the same layer. A simple feedforward NN consisting of one input, one hidden and one output layer is depicted in Fig. 3.

In vector form, the outputs of the neurons in the hidden layer can be computed as follows.

$$\mathbf{a} = \mathbf{g}(\mathbf{W}_i^T \mathbf{x} + \mathbf{b}_i) \quad (2)$$

$\mathbf{a} \triangleq [a_1 \dots a_m]^T$ represents the output vector of the hidden layer; $\mathbf{x} \triangleq [x^{(1)} \dots x^{(N_i)}]^T$ indicates the input vector of the network; $\mathbf{W}_i$ is the input weight matrix; $\mathbf{b}_i \triangleq [b_1 \dots b_m]^T$ denotes the bias vector; $\mathbf{g}(\cdot)$ is the activation function. Note that if the NN model has multiple hidden layers, the output of k^{th} hidden layer can easily be calculated similarly via replacing $\mathbf{x}$ in (2) by the output of the $(k-1)^{th}$ layer.

Besides, the output of the network is obtained by

$$\hat{\mathbf{y}} = \mathbf{W}_o^T \mathbf{a} + \mathbf{b}_o \quad (3)$$

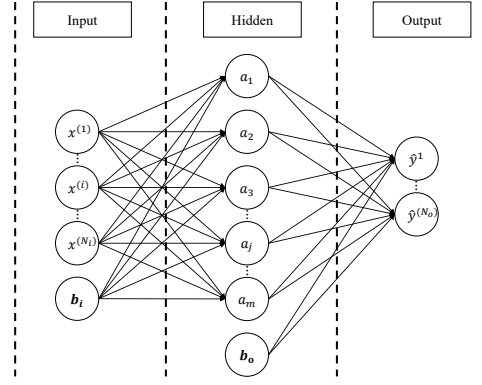


Fig. 3: A simple neural network including one input layer, one hidden layer and one output layer.

where $\hat{\mathbf{y}} \triangleq [\hat{y}_1 \dots \hat{y}_{N_o}]^T$ is the output vector; $\mathbf{W}_o$ is the output weight matrix and $\mathbf{b}_o$ is the bias vector.

The standard loss function used in the training procedure to optimize the parameters of an NN is given in (4).

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \left(y^{(i)} - f_{\theta}(x^{(i)}) \right)^2 \quad (4)$$

There are N training points and $y^{(i)}$ denotes the ground truth for the i^{th} input $x^{(i)}$. $f_{\theta}(x^{(i)})$ is the output of the NN for the corresponding input, and it is parametrized by θ which comprises of the weights and the biases of the model.

In this work, we construct a naively deep NN consisting of 2 hidden layers with respectively 16 and 8 neurons. This specific architecture was empirically observed to be sufficient regarding the classification performance obtained for training and validation data sets. The activation functions are selected as *hyperbolic tangent sigmoid* function in the hidden layers and *softmax* function in the output layer.

III. GP-ETT

First introduced in [10], GP-ETT is an effective way of tracking dynamic objects with unknown shapes. It is able to jointly estimate the kinematics and the extent of the object by using point measurements. The suggested NN-based object classification scheme basically processes the contour estimates produced by this algorithm.

In the formulation of GP-ETT, the contour of an object is represented by a radial function $r = f(\theta)$ which is to be modeled by a GP. The output of the radial function is the distance between the center and the contour at the specified polar angle. A typical shape described by radial function is shown in Fig. 4. Notice that the formulation implicitly assumes that the shape of the object is *star-convex*¹.

A noisy observation of the object contour represented by the radial function can be described as

$$\mathbf{z}_{k,l} = \mathbf{x}_k^c + \mathbf{p}(\theta_{k,l})f(\theta_{k,l}) + \mathbf{e}_{k,l}, \quad (5)$$

¹A set $\mathcal{S}(x)$ is called star-convex if each line segment from the center to any point is fully contained in $\mathcal{S}(x)$, where x denotes the position of a point.

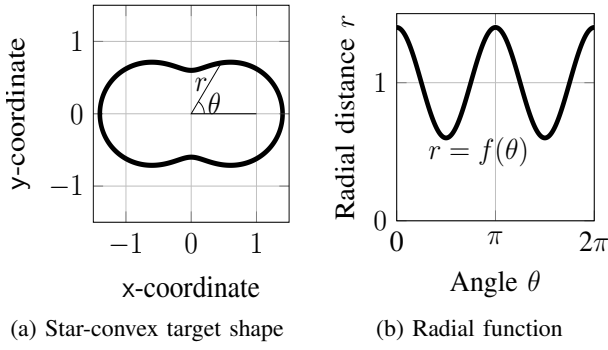


Fig. 4: An example star-convex contour described by radial function $r = f(\theta)$, [10].

where $\mathbf{x}_k^c$ is the center position of the target at time instant k , $\{\mathbf{z}_{k,l}\}_{l=1}^{n_k}$ indicate the measurements collected at time k , $\{\theta_{k,l}\}_{l=1}^{n_k}$ are the polar angles of the source points on the contour that originate the corresponding measurements, $\mathbf{e}_{k,l} \sim \mathcal{N}(0, R)$ denote i.i.d. Gaussian noise with zero mean and covariance R , and $\mathbf{p}(\theta_{k,l})$ is an orientation vector defined as $\mathbf{p}(\theta_{k,l}) \triangleq \begin{bmatrix} \cos(\theta_{k,l}) \\ \sin(\theta_{k,l}) \end{bmatrix}^\top$.

The core idea of GP-ETT is to facilitate modeling of the unknown radial function $f(\cdot)$ in (5) by a GP, i.e., $f(\theta) \sim \mathcal{GP}(\mu(\theta), k(\theta, \theta'))$ where $\mu(\cdot)$ is the mean function and $k(\cdot, \cdot)$ is the covariance function. By doing so, one can establish a probabilistic representation of the extent whose inherent spatial characteristics are conveniently encoded by the covariance function. In this work, the GP is specified with the zero mean function and the following covariance function, which is obtained by modifying the squared exponential kernel.

$$k(\theta, \theta') = \sigma_f^2 e^{-\frac{2 \sin^2(\frac{|\theta - \theta'|}{2})}{l^2}} + \sigma_r^2 \quad (6)$$

σ_f is the prior variance, l is the scaled length and σ_r essentially accounts for the uncertainty in the mean. The covariance function is illustrated in Fig. 5. Being a stationary covariance function which depends on the relative position of the inputs, the function is plotted against the difference between the input angles, i.e., $\theta - \theta'$. The figure reveals that the covariance function ensures perfect correlation between $f(\theta)$ and $f(\theta + 2\pi)$, i.e., $\rho(f(\theta), f(\theta')) = 1$, since they basically correspond to the same point on the contour. Additionally, the correlation between $f(\theta)$ and $f(\theta')$ decreases as the angle between them increases. This characteristic essentially accounts for the fact that radii at closer angles naturally tend to be more interrelated than the farther sections of the contour.

The standard GP regression requires batch processing of the measurements hence it does not apply to the online tracking and classification application due to its computational complexity and sequentially available measurements. To overcome these issues, an approximation of the standard regression was suggested to compute the posterior recursively in [10]. In particular, this method summarizes the original GP model at a

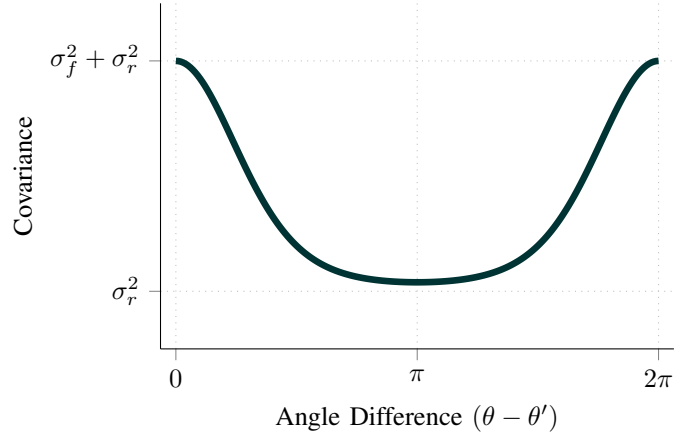


Fig. 5: The covariance function of the GP as defined in (6).

finite number of basis inputs $\mathbf{u} = [u_1 \dots u_N]^\top$, thus the posterior distribution can simply be computed by a Kalman filter regarding the following state space model.

$$\mathbf{c}_{k+1} = F\mathbf{c}_k + \mathbf{w}_k, \quad \mathbf{w}_k \sim \mathcal{N}(0, Q), \quad (7a)$$

$$z_k = H(u_k)\mathbf{c}_k + \mathbf{e}, \quad \mathbf{e} \sim \mathcal{N}(0, R(u_k)), \quad (7b)$$

$$\mathbf{c}_0 \sim \mathcal{N}(0, P_0), \quad (7c)$$

where $\mathbf{c}_k = [f(u_1) \dots f(u_N)]^\top$ is the radial function values at time k , and z_k indicates a single point measurement. Further details of the model can be found in [10].

To be able jointly estimate the target dynamics and the contour, a unified state space model is constructed which relies on the state vector, $\mathbf{x}_k \triangleq [\bar{\mathbf{x}}_k \ \mathbf{c}_k]$, where $\bar{\mathbf{x}}_k$ denotes the object dynamics including position, velocity and orientation. GP-ETT algorithm is simply implemented by an extended Kalman filter (EKF) considering this state vector. The EKF infers the joint posterior of the state, $p(\mathbf{x}_k | \mathbf{z}_{1:k})$, and the updated mean of the contour $\hat{c}$ is delivered to the classification scheme as the contour estimate.

As GP-ETT recursively updates the posterior of the state, it thereby refines the contour estimate due to the accumulation of information in time. In contrast, instant point measurements can only delineate the shape partially, and they get sparse due to occlusions or increased distance between the object and the sensor. Consequently, GP-ETT provides a more reliable basis to perform classification compared to instant measurements.

IV. SIMULATIONS AND RESULTS

In this section, the performance of the proposed algorithm is demonstrated through some simulation experiments. Throughout simulations, objects from the following shape classes are taken into consideration: circle, triangle, rectangle and plus. To form a labeled data set, first we simulate dynamic scenarios of random sized objects from each class. The initial orientation of each object is randomly selected from a uniform distribution between 0 and 2π . At each instant of the simulations, point measurements are originated from random points on the object contour with an additive Gaussian

noise whose standard deviation is set to 0.02. Subsequently, these point measurements are processed by the GP-ETT to obtain corresponding contour estimates. Contour estimates are maintained at fifty basis angles which are evenly spaced between $0-2\pi$. Some typical examples of the contour estimates from each class are exhibited in Fig. 6. The resulting data set consists of 10000 contour estimates per each shape class, and it is divided into training and test sets by the ratios of 80% and 20%, respectively. The training set is further split into two by the same ratios, 80% and 20%, to attain training and validation sets.

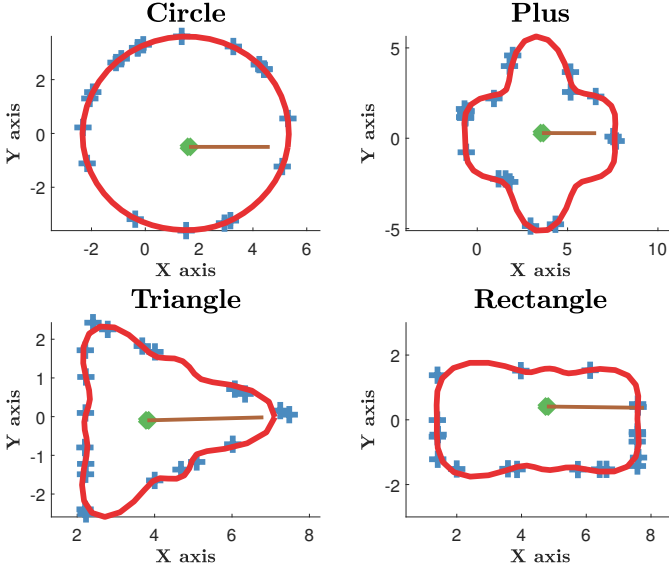


Fig. 6: Typical contour estimates produced by GP-ETT. (Contour estimates are plotted in blue while red crosses represent point measurements.)

To be able to assess the performance of the proposed algorithm in a comparative manner, we consider another classification method from our previous work, [21]. This baseline algorithm is based on a Bayesian classification scheme and is denoted as ‘BC’. Different from the proposed method, BC makes use of both the contour estimate and the associated covariance matrix, which essentially carries the local uncertainty information along the contour estimate. In particular, BC applies Unscented Transform (UT) to the outputs of GP-ETT to adopt a probabilistic representation in the feature space. Subsequently, the classification is achieved by the following Bayesian classifier which regards this probabilistic description of the feature.

$$Pr(r = i|f) = \frac{p(f|r = i)p(r = i)}{\sum_{j=1}^M p(f|r = j)p(r = j)} \quad (8)$$

where f is the feature vector; $r \in \{1, \dots, M\}$ is the class index, and $p(f|r = j)$ is the likelihood of observing the feature f from an object in class- j . The posterior probability, $Pr(r = i|f)$, can simply be computed for each class once the sufficient statistics of the class distributions are available. To this end, the feature distribution of each class is approximated

to be Gaussian, i.e., $p(f|r = i) \approx \mathcal{N}(f; \mu_i, \Sigma_i)$, and the mean, μ_i , and the covariance matrix, Σ_i , are determined by means of a supervised learning scheme using the labeled training data. For further details, see [21].

Additionally, we also implemented a variant of the proposed method which employs an NN having the same architecture; however, this time the NN is trained and tested directly on the contour estimates rather than the features extracted from these estimates. The original method is denoted as ‘NN-feature’ and ‘NN-contour’ stands for the variant.

All of the algorithms were implemented in MATLAB 2018a; in particular, we used Deep Learning Toolbox for NN-feature and NN-contour. Since the NN model is a rather shallow one, we optimized the network parameters by Levenberg-Marquardt method, which provides a fast but computationally expensive optimization scheme combining the gradient descent algorithm with Gauss-Newton method. The training procedure is stopped when the validation accuracy does not increase for the following 20 consecutive epochs. The loss function values of NN-feature at each epoch are demonstrated for both training and validation sets in Fig. 7. The training and test of the models were performed on a computer with an Intel Core i7-6700HQ CPU without using parallel programming.

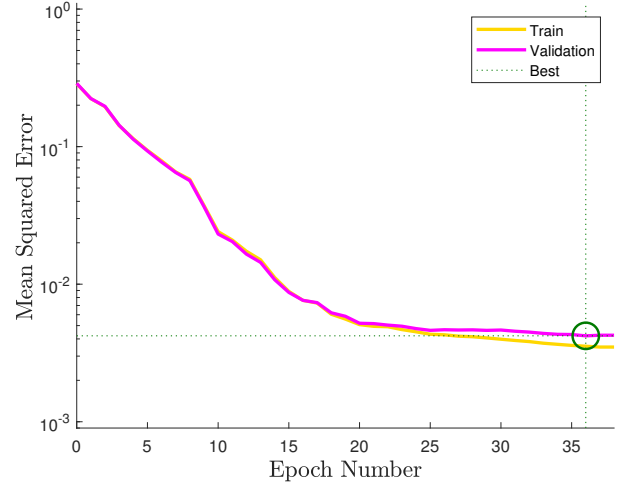


Fig. 7: The performance curves of the proposed model (NN-feature). The network parameters are set to be the values satisfying the best performance in the validation set.

The results obtained by the methods are presented in Table I. NN-feature is observed to outperform the other algorithms in the accuracy rate. Specifically, the difference in the accuracy rates of NN-feature and BC is mainly due to the test scenarios including irregular contour estimates generated due to insufficient sampling of the objects. Some typical classification outputs produced at these instants are depicted in Table II. NN-feature can handle these challenging scenarios more robustly. Even though NN-contour has a higher accuracy rate compared to BC, the execution time per object is significantly higher than the others. Particularly, NN-contour operates more slowly compared to NN-feature because the former processes fifty

contour points while the latter one considers only six features. This characteristic might render NN-contour inconvenient for object classification in real time.

A common problem encountered during the training phase of an NN is over-fitting to the training data. This leads to degradation in the generalization capabilities of the NN to other data sets. With this in mind, we briefly examined the effect of $\mathcal{L}_2$ regularization on the performance of NN-feature and NN-contour. For both of the NN-based algorithms, regularization did not improve the accuracy rate of the corresponding model.

It can be concluded that utilization of an NN in the classifier yields more robust performance especially for irregular contour estimates, which is frequently encountered in real-world scenarios when the tracked object is occluded by its surroundings. However, the simplicity of the Bayesian classifier shows its strength at the execution time by being approximately three times faster than the proposed method.

V. CONCLUSION AND FUTURE WORK

Classifying objects based on their extent estimates is still at very early ages in the ETT literature. In this work, we propose to combine a well-known deep learning algorithm with a GP based extended target tracker to classify the type of dynamic objects. To this end, various shape features and different structures of NNs are examined. The performance of the suggested algorithm is comparatively demonstrated against a Bayesian classifier. NN-based classifier shows superior performance compared to the Bayesian classifier. As future work, the algorithm will be tested on real data sets considering several applications such as identification of biological cells and annotating agents in urban driving environment.

REFERENCES

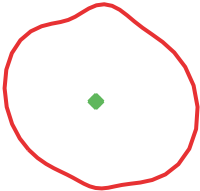
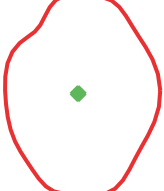
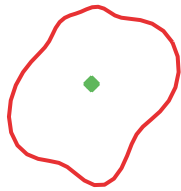
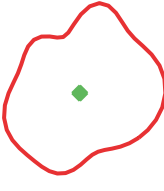
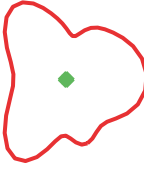
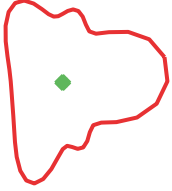
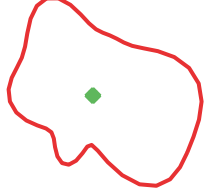
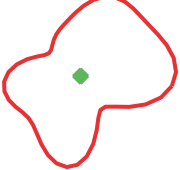
- [1] M. Baum, V. Klumpp, and U. D. Hanebeck, "A novel Bayesian method for fitting a circle to noisy points," in *13th International Conference on Information Fusion (FUSION)*. IEEE, 2010, pp. 1–6.
- [2] K. Granström, C. Lundquist, and U. Orguner, "Tracking rectangular and elliptical extended targets using laser measurements," in *14th International Conference on Information Fusion (FUSION)*. IEEE, 2011, pp. 1–8.
- [3] K. Granström and C. Lundquist, "On the use of multiple measurement models for extended target tracking," in *16th International Conference on Information Fusion (FUSION)*. IEEE, 2013, pp. 1534–1541.
- [4] J. W. Koch, "Bayesian approach to extended object and cluster tracking using random matrices," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 44, no. 3, pp. 1042–1059, 2008.
- [5] M. Feldmann, D. Franken, and W. Koch, "Tracking of extended objects and group targets using random matrices," *IEEE Transactions on Signal Processing*, vol. 59, no. 4, pp. 1409–1420, 2011.

- [6] U. Orguner, "A variational measurement update for extended target tracking with random matrices," *IEEE Transactions on Signal Processing*, vol. 60, no. 7, pp. 3827–3834, 2012.
- [7] J. Lan and X. R. Li, "Tracking of extended object or target group using random matrix—Part I: New model and approach," in *15th International Conference on Information Fusion (FUSION)*. IEEE, 2012, pp. 2177–2184.
- [8] M. Baum and U. D. Hanebeck, "Random hypersurface models for extended object tracking," in *International Symposium on Signal Processing and Information Technology*. IEEE, 2009, pp. 178–183.
- [9] —, "Shape tracking of extended objects and group targets with star-convex RHMs," in *14th International Conference on Information Fusion (FUSION)*. IEEE, 2011, pp. 1–8.
- [10] N. Wahlström and E. Özkan, "Extended target tracking using Gaussian processes," *IEEE Transactions on Signal Processing*, vol. 63, no. 16, pp. 4165–4178, 2015.
- [11] E. Özkan, N. Wahlström, and S. J. Godsill, "Rao-Blackwellised particle filter for star-convex extended target tracking models," in *19th International Conference on Information Fusion (FUSION)*. IEEE, 2016, pp. 1193–1199.
- [12] M. Kumru and E. Özkan, "3D extended object tracking using recursive Gaussian processes," in *21st International Conference on Information Fusion (FUSION)*. IEEE, 2018, pp. 1–8.
- [13] Y. Bar-Shalom, P. K. Willett, and X. Tian, *Tracking and data fusion*. YBS publishing, 2011.
- [14] S. Blackman and R. Popoli, *Design and analysis of modern tracking systems*. Artech house, 1999.
- [15] N. J. Gordon, S. Maskell, and T. Kirubarajan, "Efficient particle filters for joint tracking and classification," in *Signal and Data Processing of Small Targets*, vol. 4728. International Society for Optics and Photonics, 2002, pp. 439–450.
- [16] S. Challa and G. W. Pulford, "Joint target tracking and classification using radar and ESM sensors," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 37, no. 3, pp. 1039–1055, 2001.
- [17] P. Smets and B. Ristic, "Kalman filter and joint tracking and classification based on belief functions in the TBM framework," *Information Fusion*, vol. 8, no. 1, pp. 16–27, 2007.
- [18] X. R. Li, "Optimal Bayes joint decision and estimation," in *10th International Conference on Information Fusion (FUSION)*. IEEE, 2007, pp. 1–8.
- [19] Y. Liu and X. R. Li, "Recursive joint decision and estimation based on generalized Bayes risk," in *14th International Conference on Information Fusion (FUSION)*. IEEE, 2011, pp. 1–8.
- [20] W. Cao, J. Lan, and X. R. Li, "Extended object tracking and classification using radar and ESM sensor data," *IEEE Signal Processing Letters*, vol. 25, no. 1, pp. 90–94, 2018.
- [21] B. Tuncer, M. Kumru, E. Özkan, and A. A. Alatan, "Extended object tracking and shape classification," in *21st International Conference on Information Fusion (FUSION)*. IEEE, 2018, pp. 1–5.
- [22] H.-K. Kim and J.-D. Kim, "Region-based shape descriptor invariant to rotation, scale and translation," *Signal Processing: Image Communication*, vol. 16, no. 1-2, pp. 87–93, 2000.
- [23] L. J. Latecki, R. Lakamper, and T. Eckhardt, "Shape descriptors for non-rigid shapes with a single closed contour," in *Conference on Computer Vision and Pattern Recognition (CVPR)*, vol. 1. IEEE, 2000, pp. 424–429.
- [24] M. Bober, "MPEG-7 visual shape descriptors," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 11, no. 6, pp. 716–719, 2001.
- [25] T. P. Wallace and P. A. Wintz, "An efficient three-dimensional aircraft recognition algorithm using normalized fourier descriptors," *Computer Graphics and Image Processing*, vol. 13, no. 2, pp. 99–126, 1980.
- [26] E. Persoon and K.-S. Fu, "Shape discrimination using Fourier descriptors," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 7, no. 3, pp. 170–179, 1977.
- [27] F. Mokhtarian and A. Mackworth, "Scale-based description and recognition of planar curves and two-dimensional shapes," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, no. 1, pp. 34–43, 1986.
- [28] F. Mokhtarian and A. K. Mackworth, "A theory of multiscale, curvature-based shape representation for planar curves," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 14, no. 8, pp. 789–805, 1992.

TABLE I: Classification results on the synthetic data set in terms of accuracy rate and the execution times for each shape.

Classifier	Accuracy Rate	Execution Time (ms/object)
BC	0.94	0.57
NN-feature	0.99	1.80
NN-contour	0.97	2.55

TABLE II: Some of the results that NN shows superior performance against Bayesian approach.

 Ground Truth: Circle Bayesian: Plus NN: Circle	 Ground Truth: Plus Bayesian: Circle NN: Plus	 Ground Truth: Plus Bayesian: Rectangle NN: Plus	 Ground Truth: Plus Bayesian: Rectangle NN: Plus
 Ground Truth: Triangle Bayesian: Rectangle NN: Triangle	 Ground Truth: Triangle Bayesian: Plus NN: Triangle	 Ground Truth: Rectangle Bayesian: Plus NN: Rectangle	 Ground Truth: Rectangle Bayesian: Triangle NN: Rectangle

- [29] C. Yang, O. Tiebe, K. Shirahama, E. Łukasik, and M. Grzegorzec, "Evaluating contour segment descriptors," *Machine Vision and Applications*, vol. 28, no. 3-4, pp. 373-391, 2017.
- [30] M. Yang, K. Kpalma, and J. Ronsin, "A survey of shape feature extraction techniques," in *Pattern Recognition*. IN-TECH, 2008, pp. 43-90.
- [31] Y. Tao, S.-C. B. Lo, M. T. Freedman, and J. Xuan, "A preliminary study of content-based mammographic masses retrieval," in *Medical Imaging 2007: Computer-Aided Diagnosis*, vol. 6514. International Society for Optics and Photonics, 2007.